



COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the *Copyright Act 1968* (the Act).

The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.



FIT3080 – Artificial Intelligence

Solving Problems as Search – Irrevocable and Adversarial Search

Sections 4.1, 6.1-6.3

LN3 Learning Objectives

- **Problem formulation**
- **Problem-solving agents**
- **Search algorithms and strategies**
 - **Tentative algorithms**
 - > Backtrack [Chapter 5.1, 5.3, 5.4]
 - > Tree and graph search **strategies**:
 - **Uninformed**: BFS, UCS, DFS, DLS, IDS [Chapter 3.1-3.4.4, 3.4.6]
 - **Informed**: Greedy best-first search, A, A* [Chapter 3.5.1-3.5.4]
 - **Irrevocable algorithms**
 - > **Informed**: Hill climbing, Local beam search, Simulated annealing, Genetic algorithms [Chapter 4.1]
- **Adversarial search algorithms [Chapter 6.1-6.3]**
 - Optimal decisions
 - Minimax, α - β pruning





FIT3080 – Artificial Intelligence

Irrevocable Search Algorithms

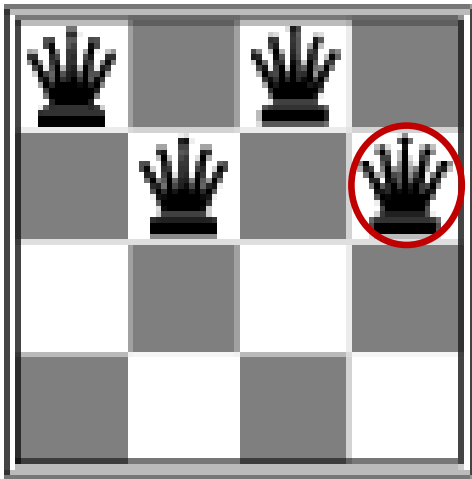
Local Search Algorithms

- **Local search algorithms**
 - keep a single “current” state, try to improve it
- **Often used for optimization problems where the goal state is the solution**
- **State space = set of “complete” configurations**
- **Find configuration satisfying constraints, e.g., n-queens problem**



Example: n -Queens Problem

- Put n queens on an $n \times n$ board with no two queens on the same row, column or diagonal





FIT3080 – Artificial Intelligence

Hill Climbing and Local Beam Search

Hill Climbing Algorithm

Procedure Hill Climbing(*current-state*)

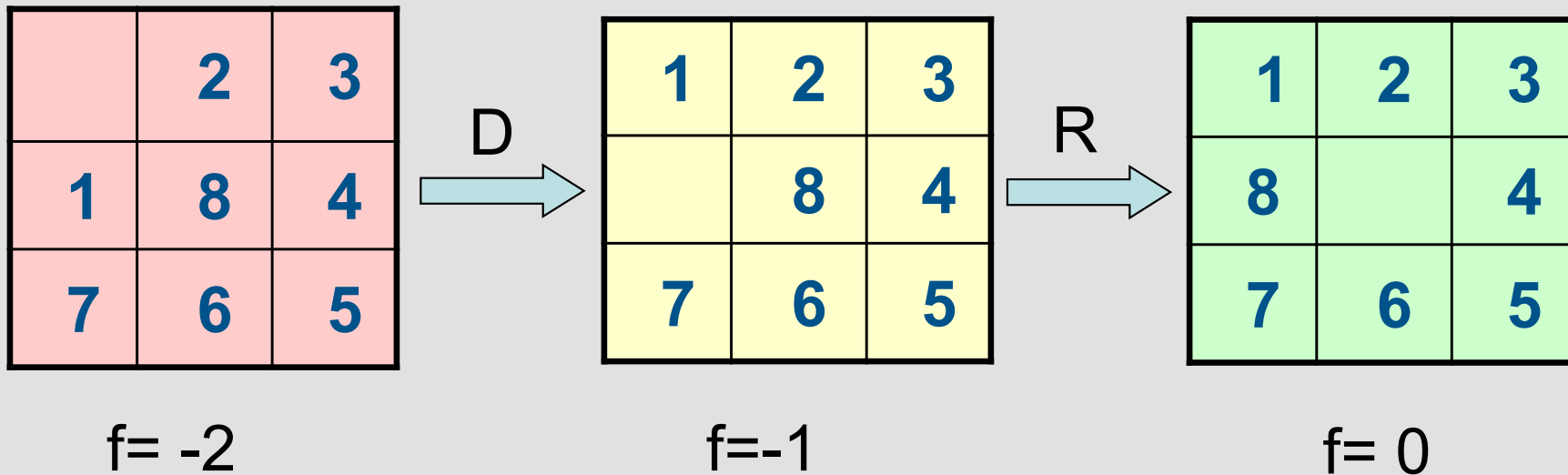
1. **If** *current-state* = *goal-state* **Then** return it
2. **Else** until a solution is found or no more operators can be applied **do**
 - a. Select an operator that has not been applied yet to *current-state* and apply it to generate *new-state*
 - b. Evaluate *new-state*:
 - i. **If** *new-state* = *goal-state* **Then** return it
 - ii. **Elseif** *new-state* is better than *current-state* **Then** *current-state* $\leftarrow$ *new-state*

Steepest ascent hill-climbing: select the best operator



Hill Climbing – Example 8 Puzzle (I)

- $f = - \{ \text{number of tiles out of place} \}$



Hill Climbing – Example 8 Puzzle (II)

- $f = - \{ \text{number of tiles out of place} \}$

Current

1	2	5
	8	4
7	6	3

$f = -2$

Goal

1	2	3
	8	4
7	6	5

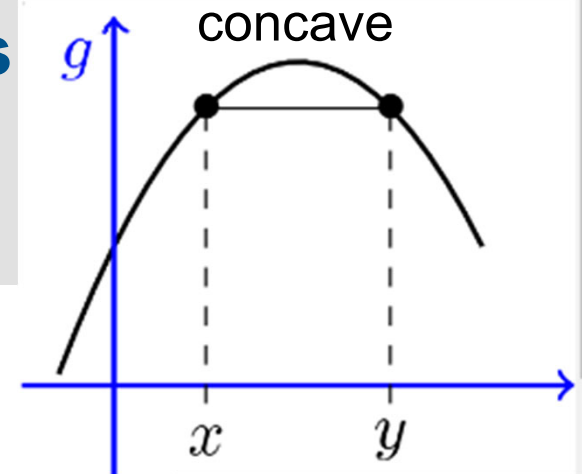
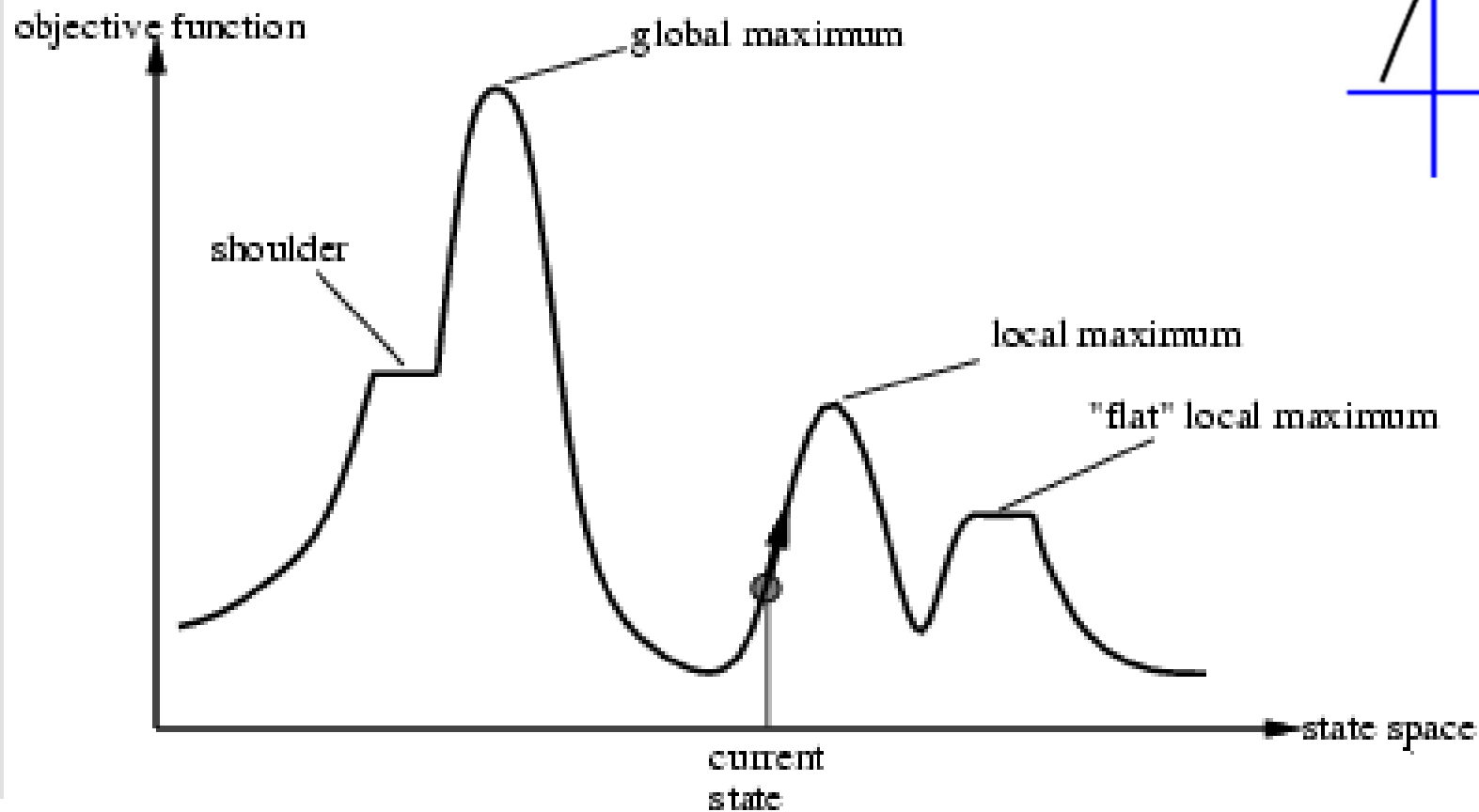
$f = 0$

Stuck in local maximum



Hill-climbing Search

- Finds the optimum for concave functions
- Problem: depending on initial state, can get stuck in local maxima



Local Beam Search

- Keep track of k states rather than just one
- Start with k randomly generated states
- At each iteration, all the successors of all k states are generated
 - If any one is a goal state **then** return it
 - **Else** select the k best successors from the complete list and repeat





FIT3080 – Artificial Intelligence

Simulated Annealing

Simulated Annealing

- Based on the physical process of annealing
- Idea: escape local maxima/minima by allowing some “bad” moves, but **gradually decrease** their frequency
- *Temperature (T)* – the temperature at which the annealing takes place
- *Annealing schedule* – the rate at which the temperature is lowered



Simulated Annealing Algorithm

Procedure Simulated Annealing(*current-state*)

1. **If** *current-state* = *goal-state* **Then** return it
2. *BestSoFar* $\leftarrow$ *current-state*
3. Initialize *T* according to the annealing schedule
4. **Until** no more operators can be applied **do**
 - a. Select an operator that has not been applied yet to *current-state* and apply it to generate *new-state*
 - b. Evaluate *new-state*. Compute:
 $\Delta E = \text{Value}(\text{current-state}) - \text{Value}(\text{new-state})$
 - i. **If** *new-state* = *goal-state* **Then** return it
 - ii. **Elseif** $\Delta E < 0$ (*new-state* is better than *current-state*) **Then**
current-state $\leftarrow$ *new-state*
If *new-state* is better than *BestSoFar* **Then** *BestSoFar* $\leftarrow$ *new-state*
 - iii. **Else** with probability $\text{Pr} = e^{-\Delta E/T}$ *current-state* $\leftarrow$ *new-state*
 - c. Revise *T* according to the annealing schedule
 - d. **If** *T* = 0 **Then** return *BestSoFar*

**Maximization
problem**

Simulated Annealing – Step 4.b.iii

- **When we get to this step, we know that $\Delta E \geq 0$**

- If $T1 > T2$, $e^{-\frac{\Delta E}{T1}} > e^{-\frac{\Delta E}{T2}}$

- ➔ The probability of adopting a worse state in T1 is higher than in T2



Properties of Simulated Annealing Search

- One can prove: If T decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1
- It is widely used in VLSI layout and airline scheduling





FIT3080 – Artificial Intelligence

Genetic Algorithms

Genetic Algorithms

- Start with a **population** of k randomly generated states
- A state (**chromosome**) is represented as a string over a finite alphabet of **genes** (often a string of 0s and 1s)
- A successor state is generated by combining two parent states
- Evaluation function (**fitness function**):
 - Higher values for better states
- Produce the next generation of states by **selection, crossover and mutation**

GAs: Example 8-Queens Problem (I)

Initial population Fit func. Prob select.

24748552	24	31%
32752411	23	29%
24415124	20	26%
32543213	11	14%

chromosome

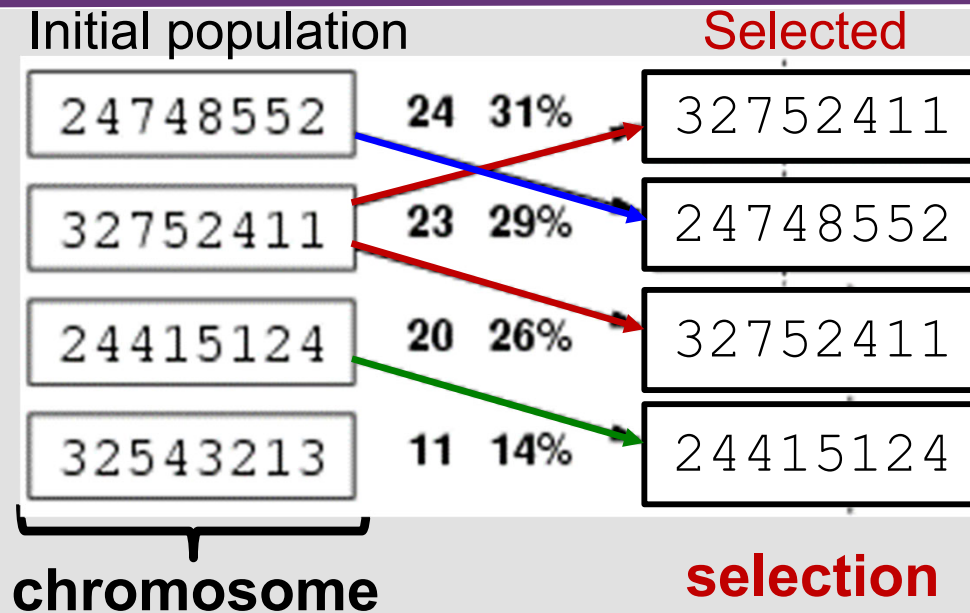
- **Representation:**

- **Chromosome:** 1 gene per column (8 genes per chromosome)
- **Gene:** row # (between 1 and 8) of the queen that is in column i

- **Fitness function: number of non-attacking pairs of queens (min = 0, max = $8 \times 7/2 = 28$)**

- **Probability of selection:** $\frac{24}{24+23+20+11}=31\%$, $\frac{23}{24+23+20+11}=29\%$

GAs: Example 8-Queens Problem (II)



- **Representation:**

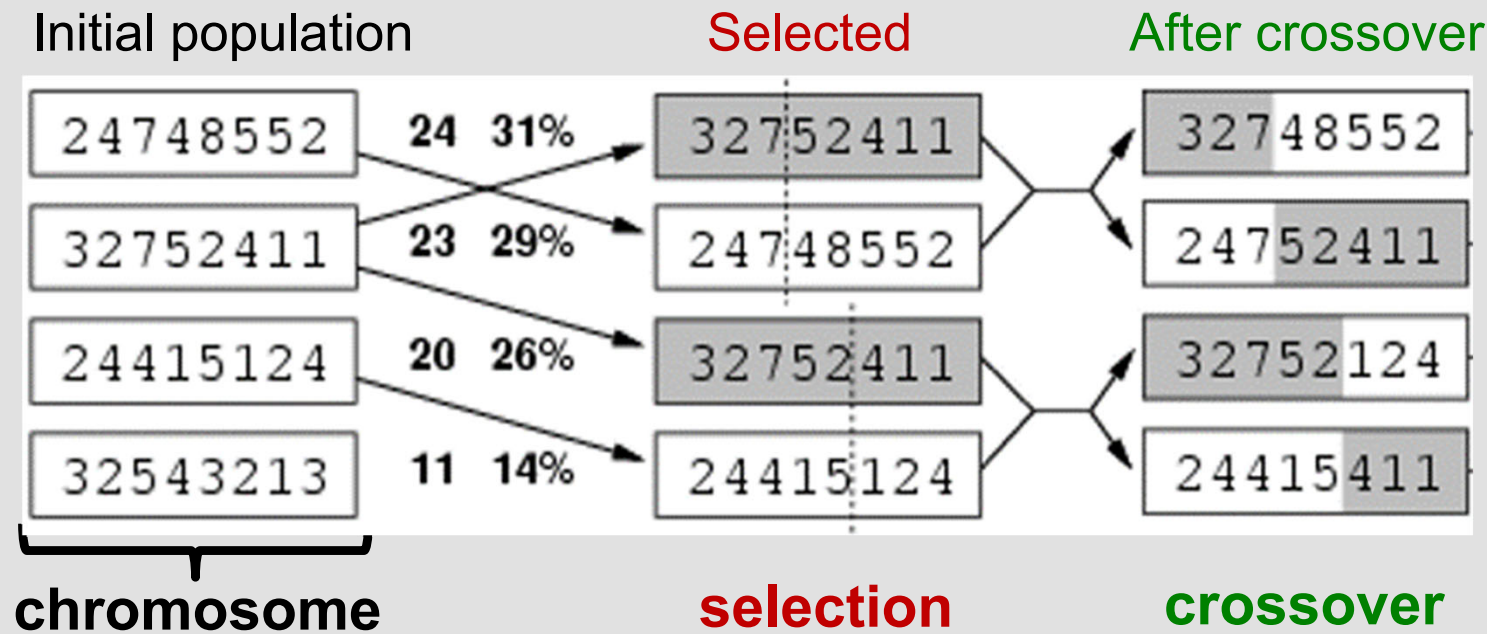
- **Chromosome:** 1 gene per column (8 genes per chromosome)

- **Gene:** row # (between 1 and 8) of the queen that is in column i

- **Fitness function: number of non-attacking pairs of queens (min = 0, max = $8 \times 7/2 = 28$)**

- **Probability of selection:** $\frac{24}{24+23+20+11} = 31\%$, $\frac{23}{24+23+20+11} = 29\%$

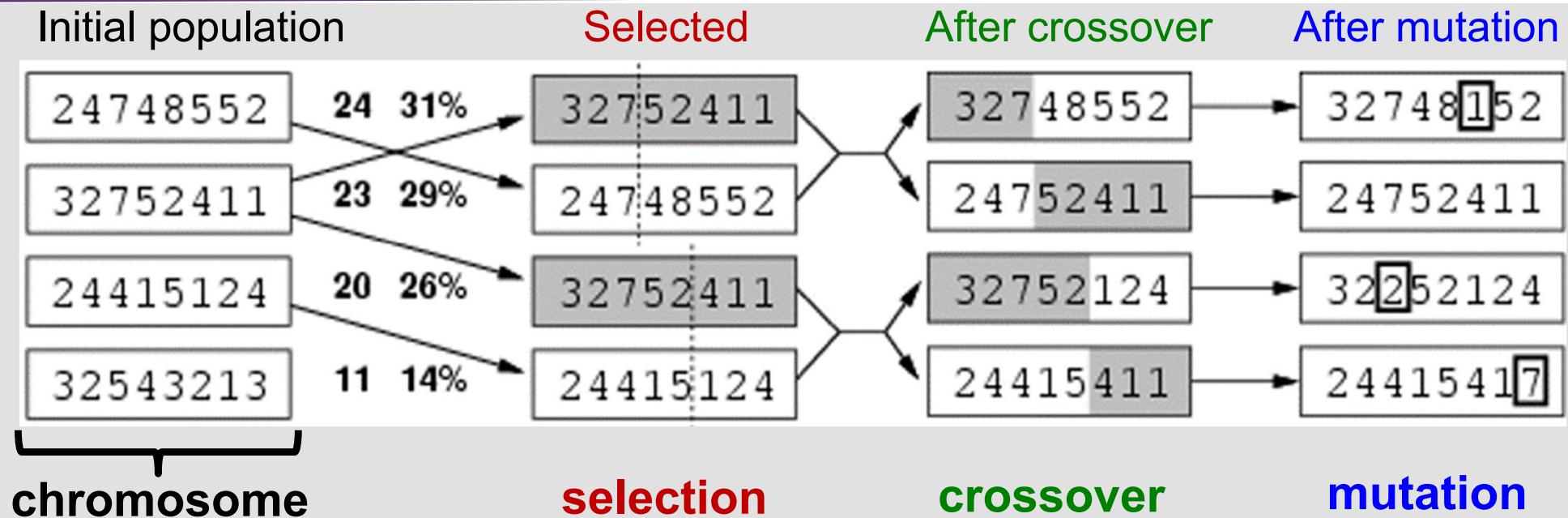
GAs: Example 8-Queens Problem (III)



- **Crossover: crossover point is selected probabilistically**

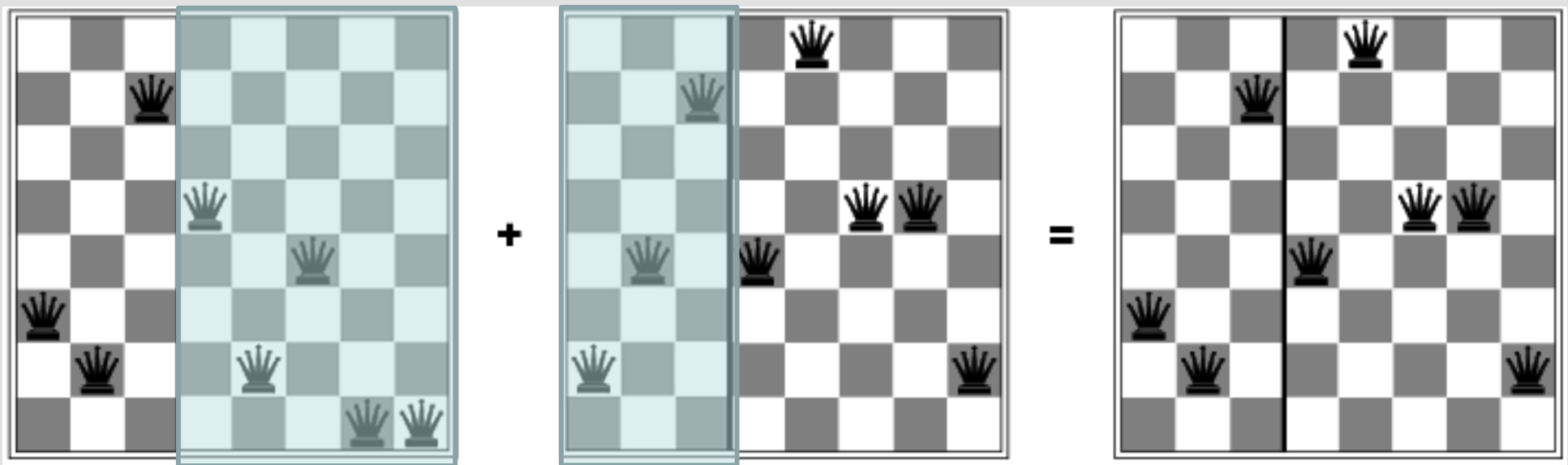


GAs: Example 8-Queens Problem (IV)



- **Mutation: mutated genes are selected probabilistically**
- **Some operations may be skipped**
- **The operations can be performed in a different order**

GA Crossover: Example 8-Queens Problem



Search Algorithms – A Perspective

Graph/Tree search

Greedy BestFS,
A, A*, IDA*, WA*

BFS, UCS,
DFS, DLS, IDS

Backtrack

Search algorithms

Hill climbing

Simulated annealing

Genetic algorithms

- **Informedness in Graphsearch depends on g and h**

- A $f(n) = g(n) + h(n)$ ($g(n) \geq g^*(n), 0 \leq h(n)$)
- A* ($g(n) \geq g^*(n), 0 \leq h(n) \leq h^*(n)$)

- **Uninformed Graphsearch**

- BFS $\in$ A* when $g(n) = \text{depth}$ and $h(n) = 0$
- UCS $\in$ A* with $g(n) \geq 0$ and $h(n) = 0$
- DFS, DLS, IDS $\in$ Graphsearch, DFS, DLS, IDS $\notin$ A

- **Informed Graphsearch**

- Greedy best-first search with $g(n) = 0$ and $h(n) \geq 0 \notin$ A





FIT3080 – Artificial Intelligence

Adversarial Search Algorithms

Assumptions about the Environment

- **Fully observable**
- **Known**
- **Multi agent**
- **Deterministic**
- **Sequential**
- **Static**
- **Discrete**



Adversarial Search – Chess

- **Gary Kasparov vs Deep Blue (1997)**
 - Approx. 10^{40} states



A short recap from the BBC:
<https://www.youtube.com/watch?v=KF6sLCeBj0s>

Adversarial Search – Go

- **Lee Sedol vs AlphaGo (2016)**

- Approx. 10^{170} states



Documentary about this match:

<https://www.youtube.com/watch?v=WXuK6gekU1Y>

Types of Games

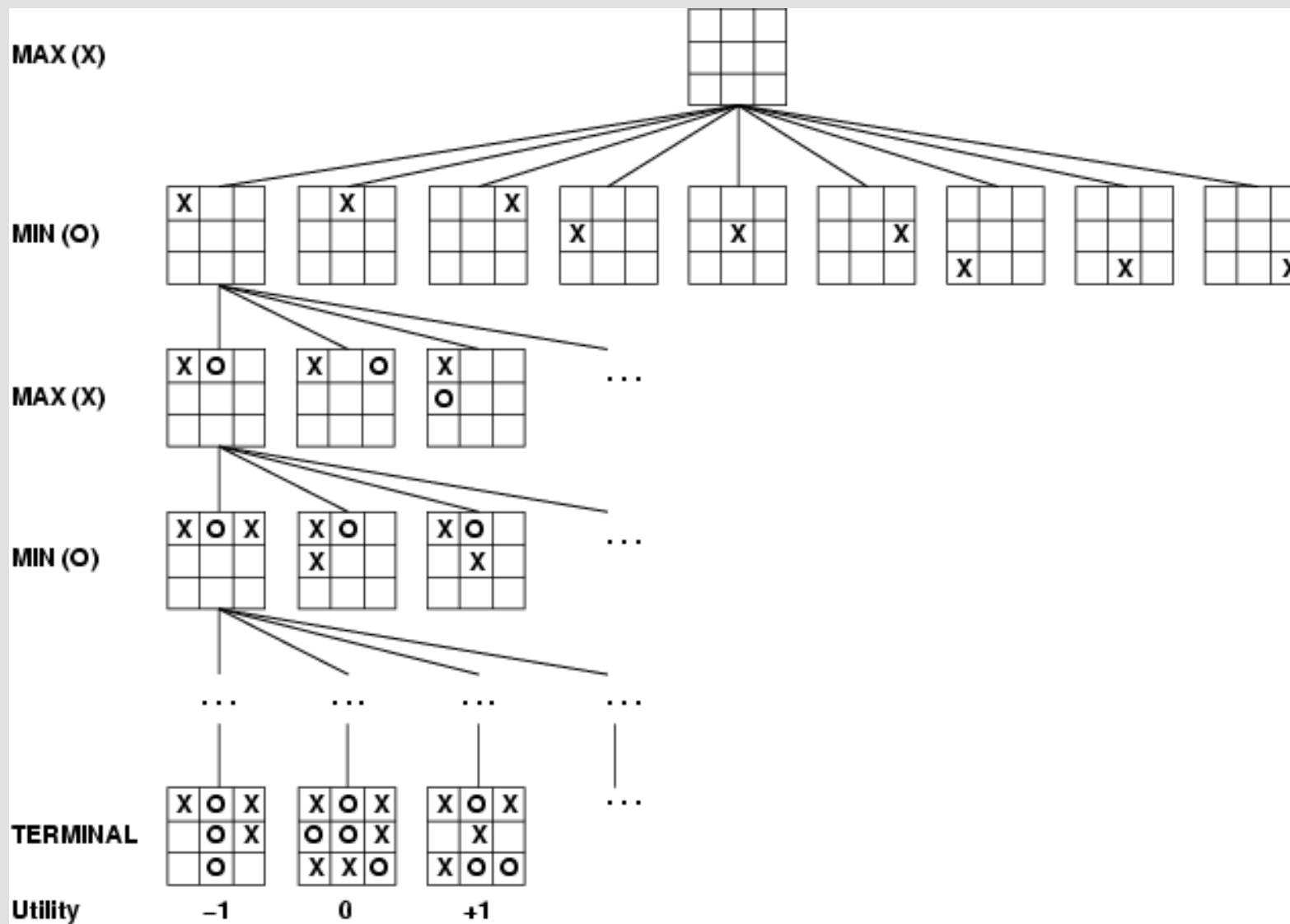
- **Perfect vs imperfect information**
 - **Perfect** (deterministic, fully observable) – chess, go, noughts-and-crosses (tic-tac-toe), draughts (checkers)
 - **Imperfect** (stochastic, partially observable) – poker, bridge, backgammon
- **Multi-agent: n-Players (e.g., $n = 2$ for chess)**
- **Sequential vs simultaneous**
- **Turn-based vs real-time**
- **Zero or constant sum vs. non-zero sum games**

We focus on 2-player, sequential, perfect information, zero-sum games

Searching Game Trees – Conventions

- **Players are MAX and MIN**
 - A position favourable to MAX has a value > 0 (often ∞)
 - A position favourable to MIN has a value < 0 (often $-\infty$)
- **Goal: find a winning strategy for MAX**
 - MAX prefers maximum utility
 - MIN prefers minimum utility

Example Game Tree – Tic-tac-toe



Game Trees – Concepts

- Initial configuration of the board/game at the root node
- Each level of the tree is called a ply
- Each successive ply is associated with the alternating player
- Each player chooses the most promising move for themselves
- Each move adds new nodes to the frontier
- Every node is associated with a *minimax* value or utility
- Leaves are called terminal and correspond to win, loss or draw according to their utility

Solving Game Trees

- **A solution is a game tree where we can label the root node with an outcome (win, loss, draw, or a utility value)**
- **Each move available to the opponent is a sub-problem (another game)**
- **All sub-problems must be solved before the parent node can be considered solved**

Game Trees – Winning Strategy

- **Goal: find a winning strategy for MAX**
 - For all nodes representing a game situation where it is MIN's move next, show that MAX can win from **every** position to which MIN might move
 - For all nodes representing a game situation where it is MAX's move next, show that MAX can win from **just one** position to which MAX might move

Games versus Search Problems

- **Unpredictable opponent → must specify a move for every possible opponent reply**
- **Time limits: not all games can be searched to the end → find a good first move**



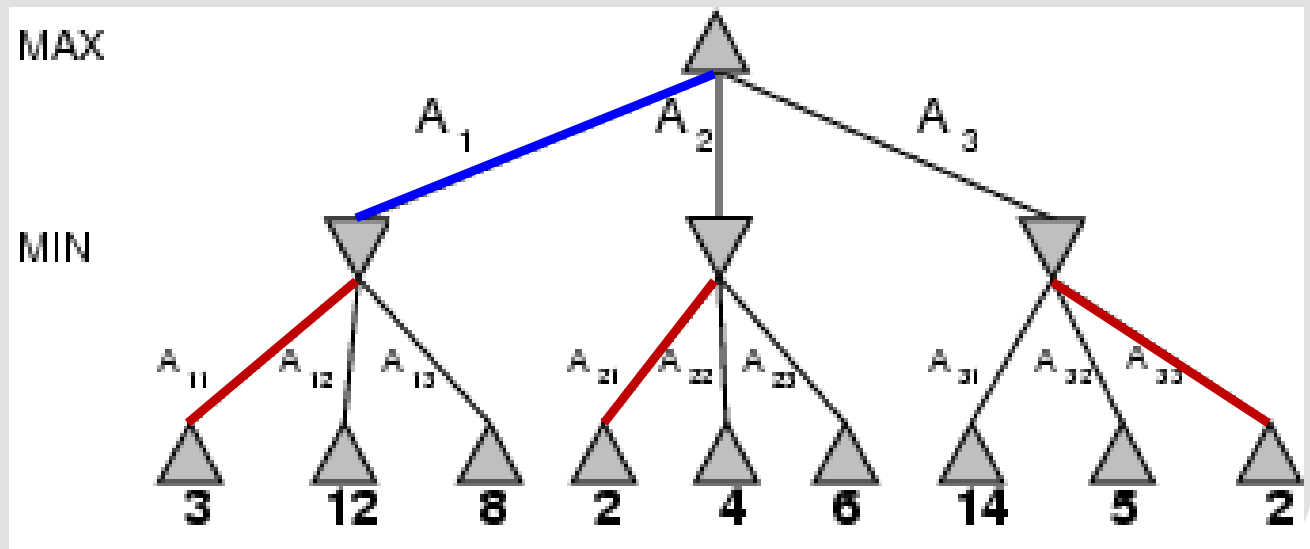


FIT3080 – Artificial Intelligence

Minimax Search

Minimax Ideas

- If MAX were to choose among terminal nodes, they would take the node with the largest value
- If MIN were to choose among terminal nodes, they would take the node with the smallest value
- Choose move to the position with highest **minimax value**: best achievable payoff against best play
- E.g., 2-ply game:



Minimax Algorithm – Definitions

- **S_0** : the initial state of the game
- **$ACTIONS(s)$** : set of moves applicable in state s
- **$RESULT(s, a)$** : the next state after applying action a in state s
- **$IS-TERMINAL(s)$** : tests whether we need to stop going down the search tree
- **$UTILITY(s)$** : payoff in state s for the current player

Minimax Algorithm

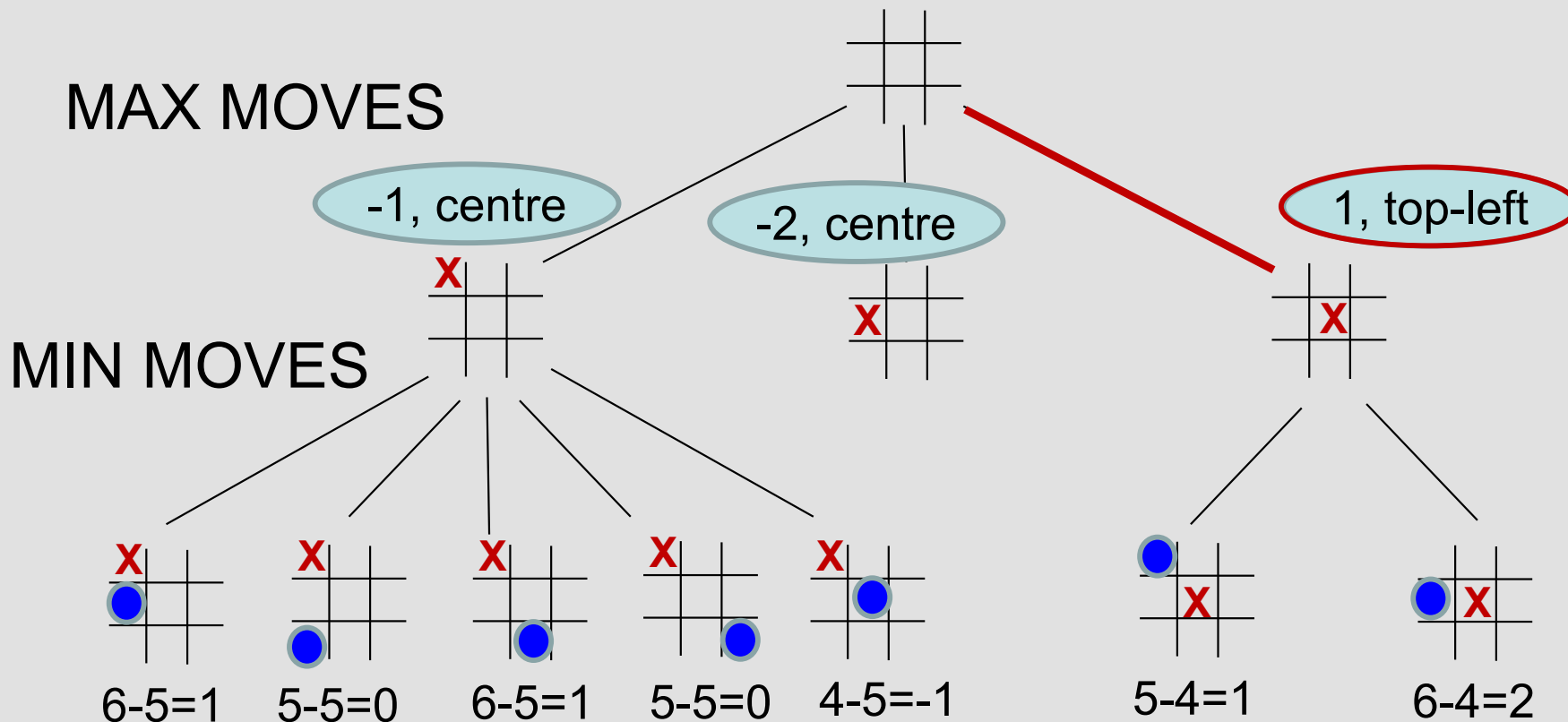
```
1: function MINIMAX-SEARCH(state)
2:   value, move  $\leftarrow$  MAX-VALUE(state)
3:   return move
4: function MAX-VALUE(state)
5:   if IS-TERMINAL(state) then return UTILITY(state), null
6:   v, move  $\leftarrow -\infty$ , null
7:   for a  $\in$  ACTIONS(state) do
8:     v2, a2  $\leftarrow$  MIN-VALUE(RESULT(state, a))
9:     if v2 > v then
10:      v, move  $\leftarrow v2, a$ 
11:   return v, move
12: function MIN-VALUE(state)
13:   if IS-TERMINAL(state) then return UTILITY(state), null
14:   v, move  $\leftarrow \infty$ , null
15:   for a  $\in$  ACTIONS(state) do
16:     v2, a2  $\leftarrow$  MAX-VALUE(RESULT(state, a))
17:     if v2 < v then
18:      v, move  $\leftarrow v2, a$ 
19:   return v, move
```



Minimax Example: Tic-Tac-Toe

- Evaluation function:**

$\{ \# \text{ of rows, columns, diagonals available to MAX } \} -$
 $\{ \# \text{ of rows, columns, diagonals available to MIN } \}$



Properties of Minimax

Minimax implements Complete Depth First Exploration

- All paths are explored to the maximum depth m
- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ (DFS-style exploration)
 $O(m)$ (Backtrack-style exploration)

For chess, $b \approx 35$, $m \approx 100$ for “reasonable” games
→ exact solution completely infeasible





FIT3080 – Artificial Intelligence

Alpha-Beta Pruning

Definitions: α and β Values

- **Nilsson:**

- α -value of a MAX node is the current largest utility backed up from its successors
- β -value of a MIN node is the current lowest utility backed up from its successors

- **Russell and Norvig:**

- α = the value of the best (highest-value) choice we have found so far at any choice point along the path for MAX
- β = the value of the best (lowest-value) choice we have found so far at any choice point along the path for MIN

- **Both**

- α -value is the lower bound for a MAX value
- β -value is the upper bound for a MIN value

Rules for Pruning Nodes (Nilsson)

- **α cut-off:** search can be discontinued below any MIN node having a β -value $\leq \alpha$ -value of *any* of its MAX node ancestors
 - The final backed-up value of this MIN node is set to its β -value
- **β cut-off:** search can be discontinued below any MAX node having an α -value $\geq \beta$ -value of *any* of its MIN node ancestors
 - The final backed-up value of this MAX node is set to its α -value

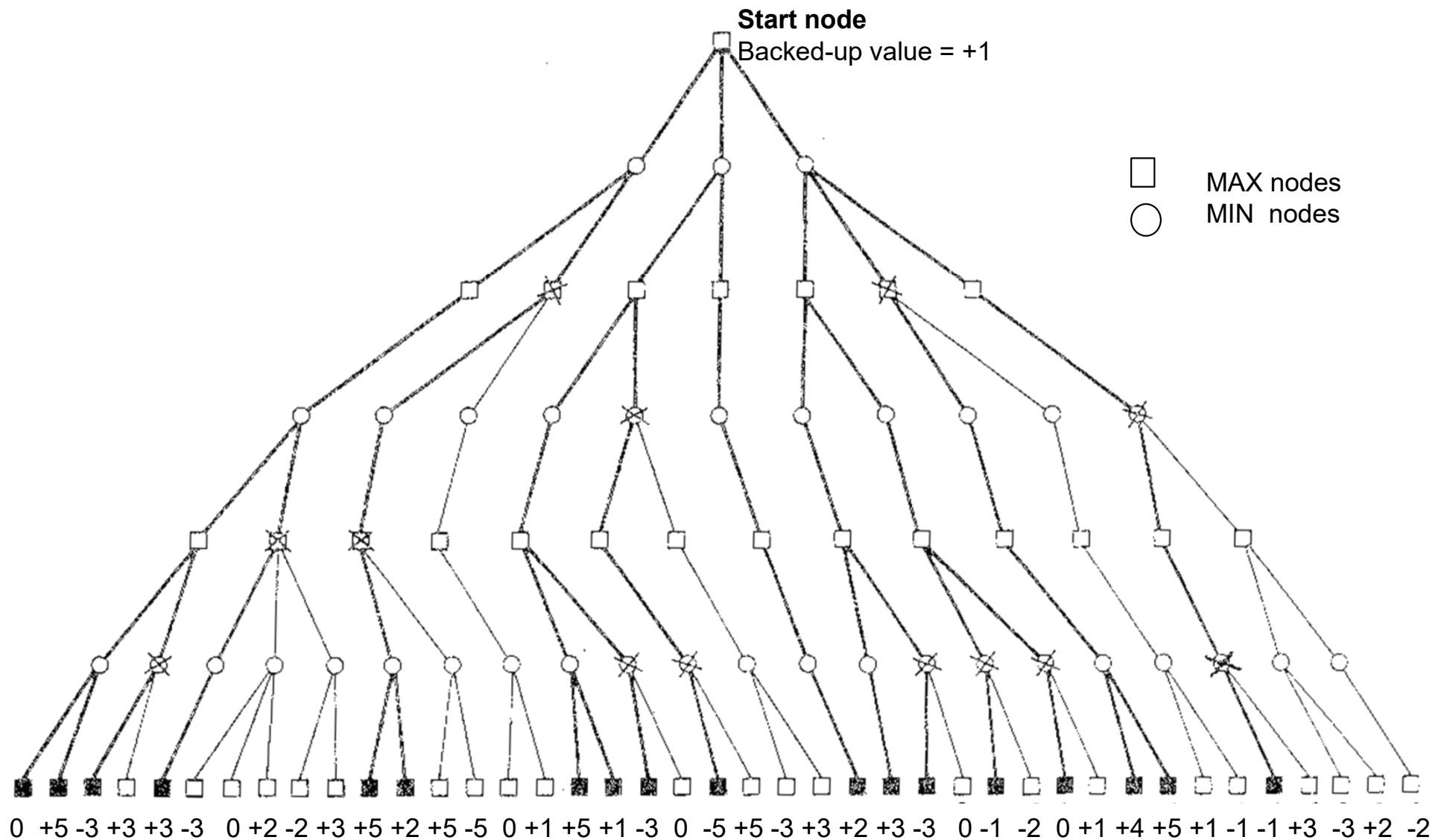


Termination Condition

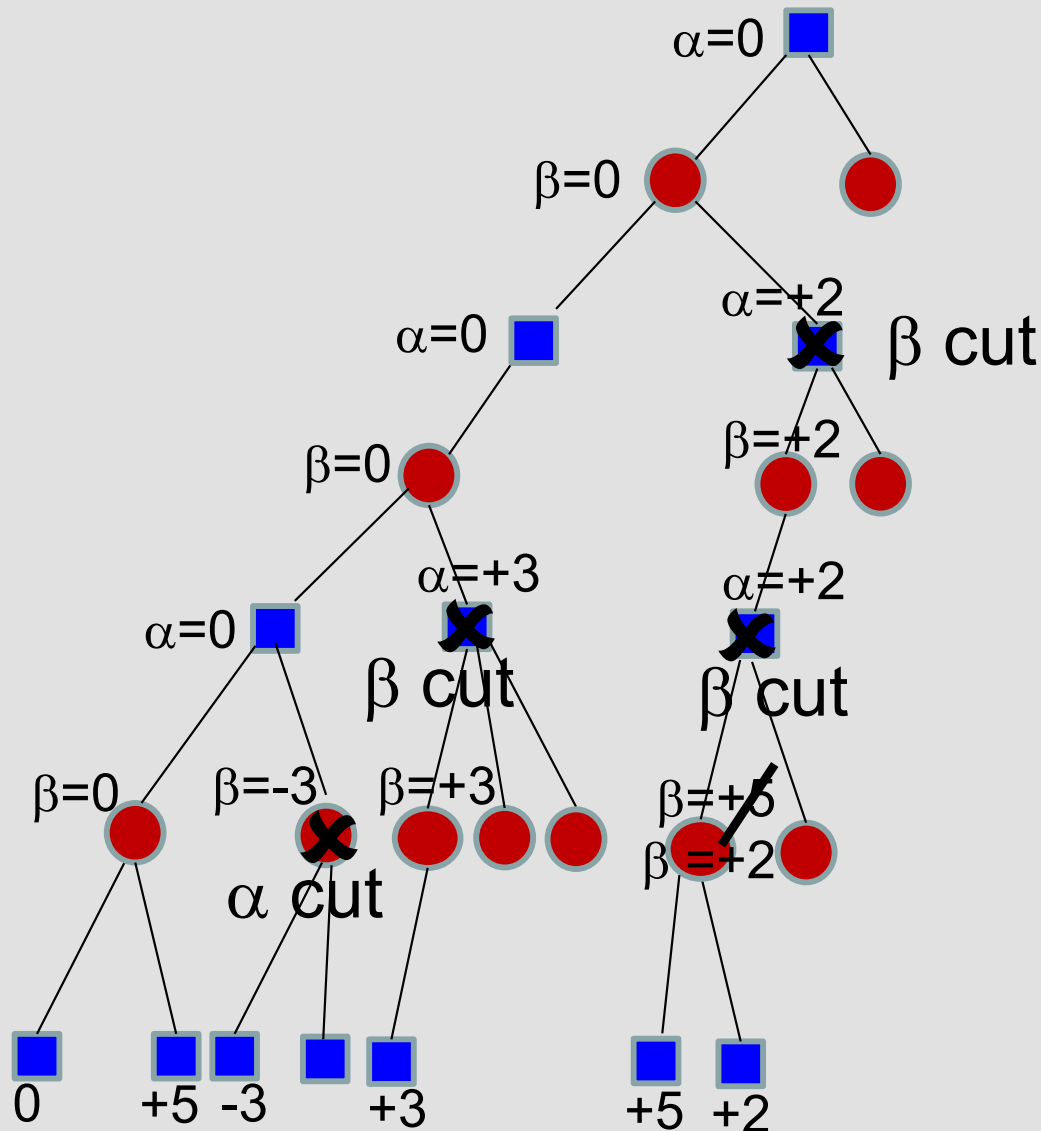
- **All the successors of the start node are given final backed-up values**
- **The best first move is that which creates the successor with the highest value**



α - β Pruning – Large Example



α - β Pruning – Part of Large Example



Alpha-Beta Search

```
1: function ALPHA-BETA-SEARCH(state)
2:   value, move  $\leftarrow$  MAX-VALUE(state,  $-\infty$ ,  $\infty$ )
3:   return move
4: function MAX-VALUE(state,  $\alpha$ ,  $\beta$ )
5:   if IS-TERMINAL(state) then return UTILITY(state), null
6:   v, move  $\leftarrow -\infty$ , null
7:   for a  $\in$  ACTIONS(state) do
8:     v2, a2  $\leftarrow$  MIN-VALUE(RESULT(state, a),  $\alpha$ ,  $\beta$ )
9:     if v2 > v then
10:      v, move  $\leftarrow$  v2, a
11:       $\alpha \leftarrow \max(\alpha, v)$ 
12:      if  $v \geq \beta$  then return v, move
13:   return v, move
14: function MIN-VALUE(state,  $\alpha$ ,  $\beta$ )
15:   if IS-TERMINAL(state) then return UTILITY(state), null
16:   v, move  $\leftarrow \infty$ , null
17:   for a  $\in$  ACTIONS(state) do
18:     v2, a2  $\leftarrow$  MAX-VALUE(RESULT(state, a),  $\alpha$ ,  $\beta$ )
19:     if v2 < v then
20:      v, move  $\leftarrow$  v2, a
21:       $\beta \leftarrow \min(\beta, v)$ 
22:      if  $v \leq \alpha$  then return v, move
23:   return v, move
```

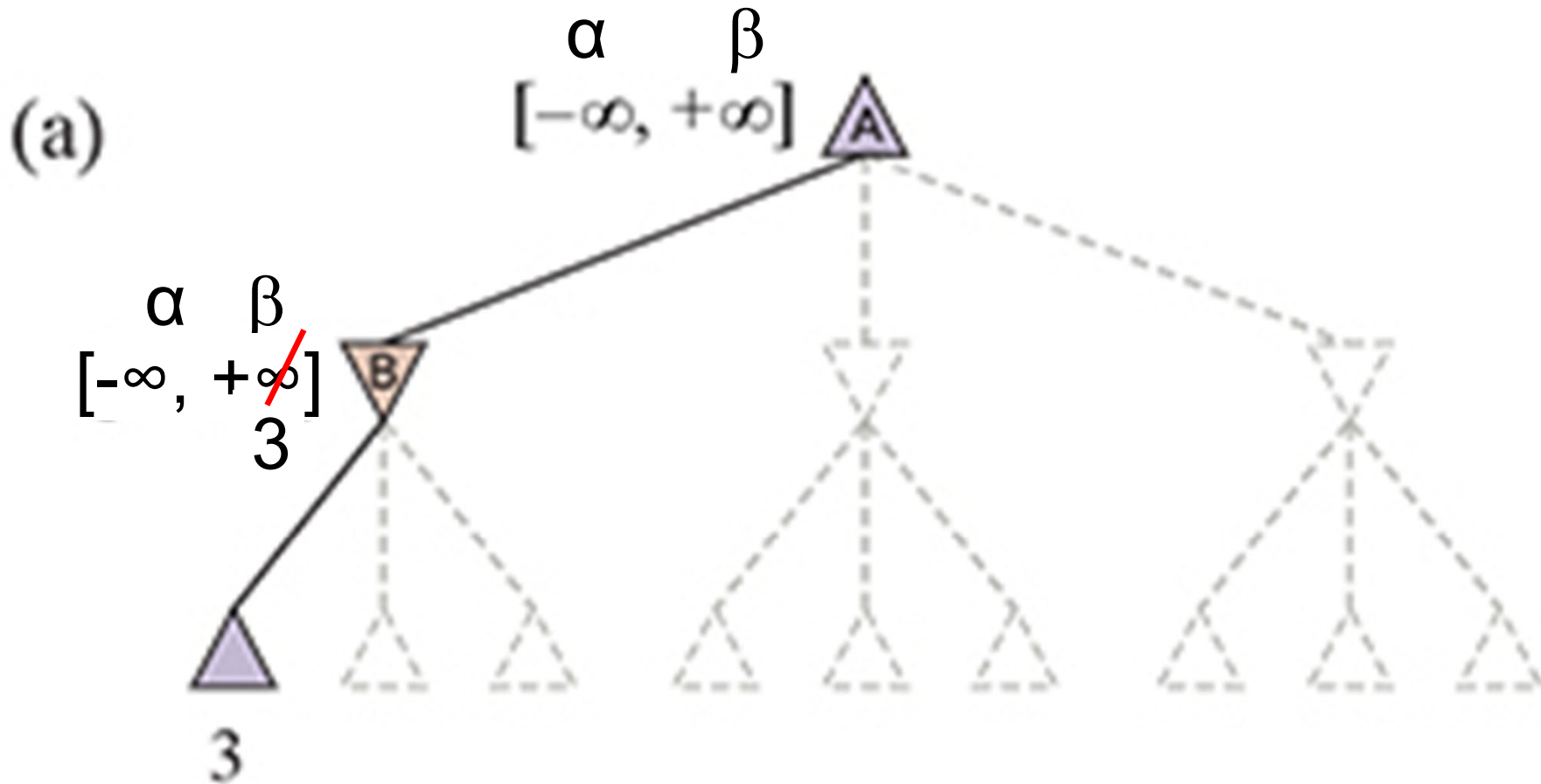
β cut-off – if
not last child

α cut-off – if
not last child

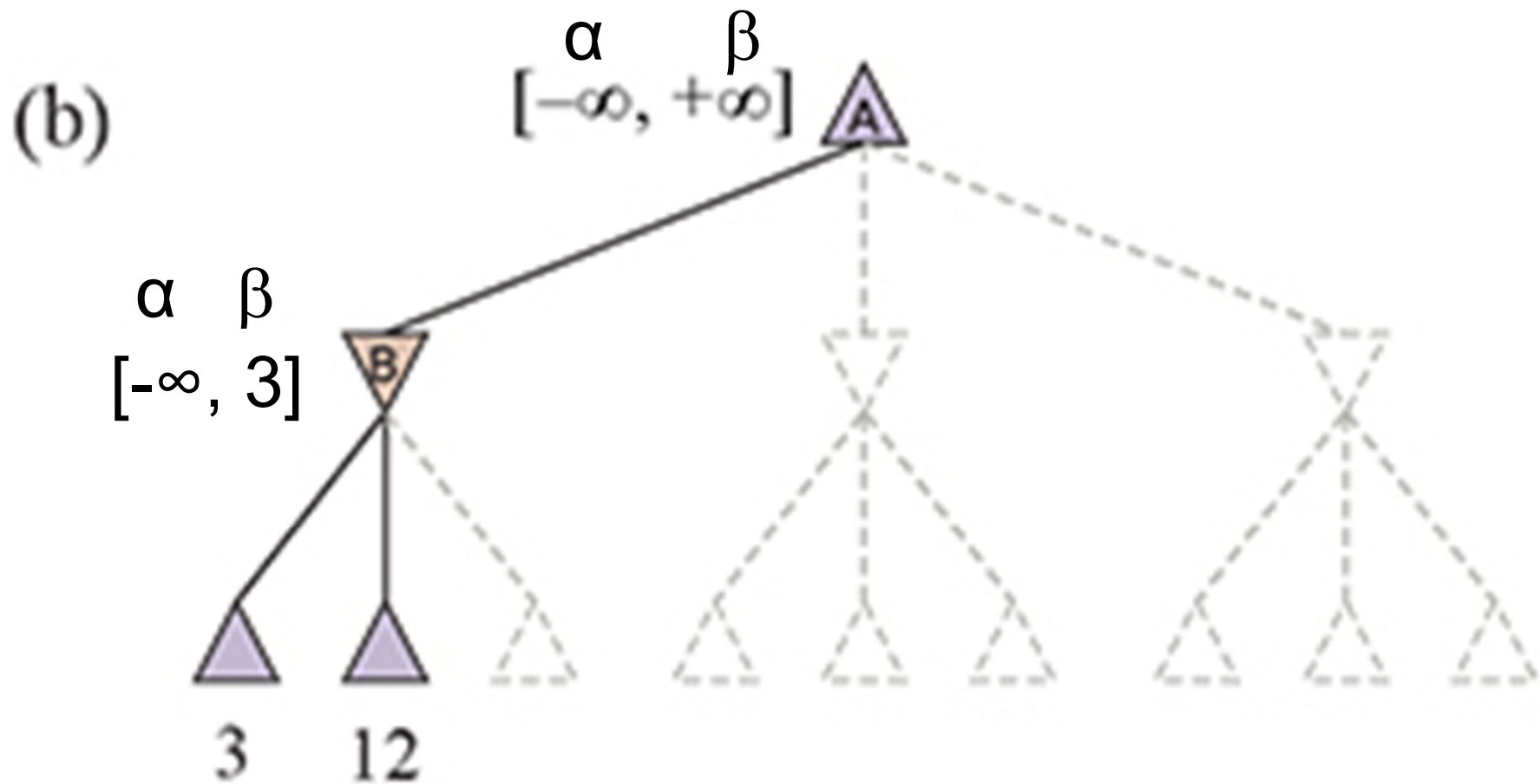
Simulations can be
found here:
[https://schaerli.org/in
fo2/abTreePractice/](https://schaerli.org/in
fo2/abTreePractice/)



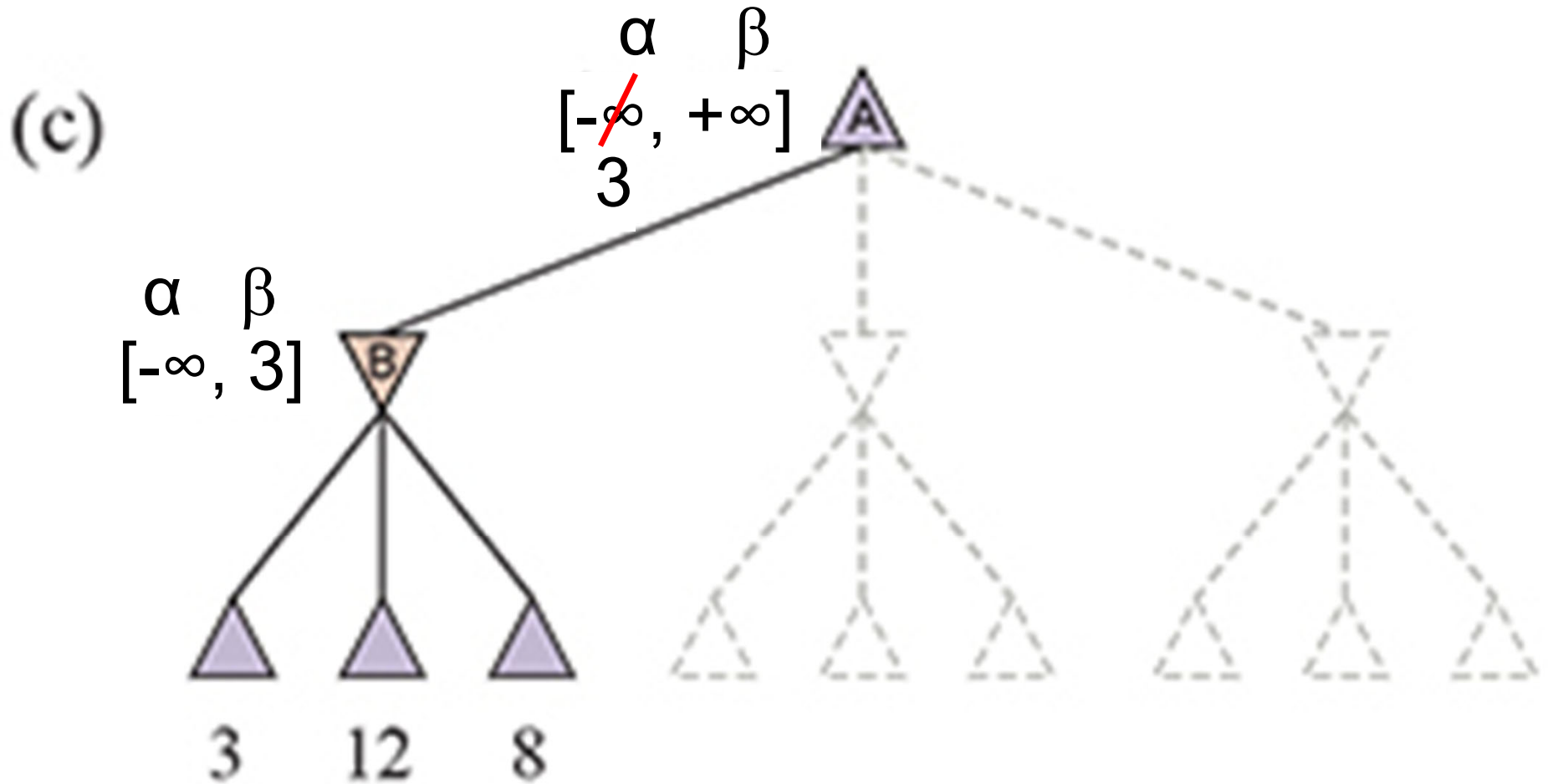
Alpha-Beta Search – Example (I)



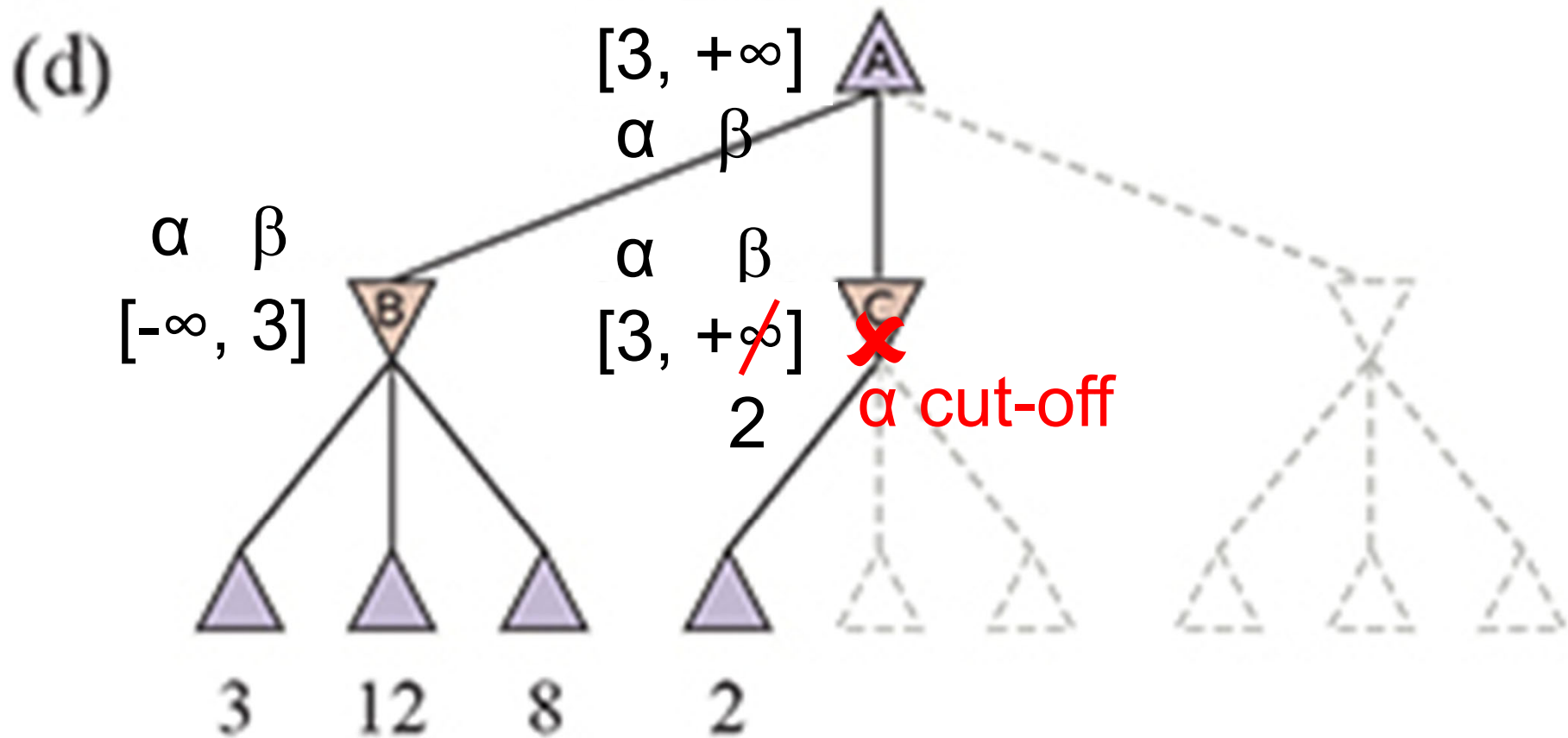
Alpha-Beta Search – Example (II)



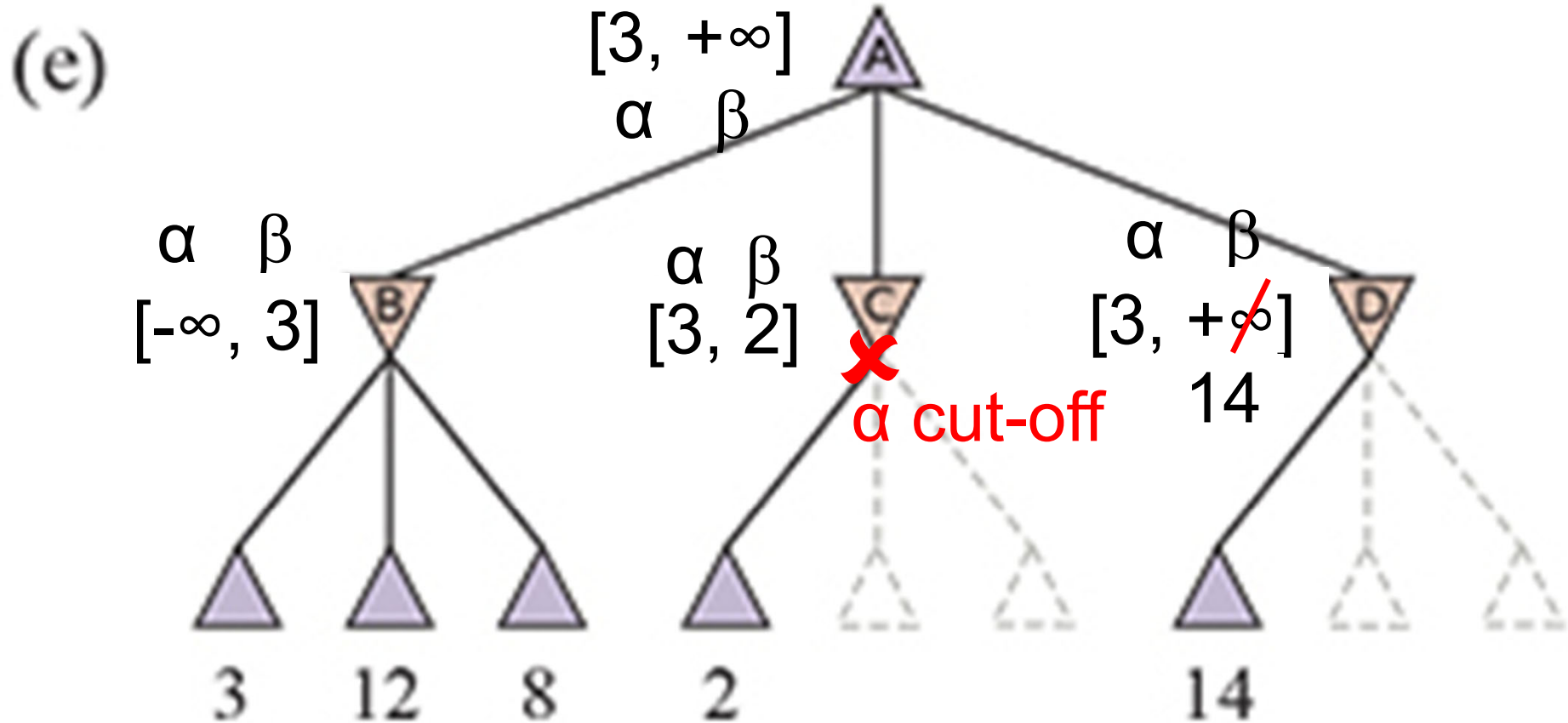
Alpha-Beta Search – Example (III)



Alpha-Beta Search – Example (IV)

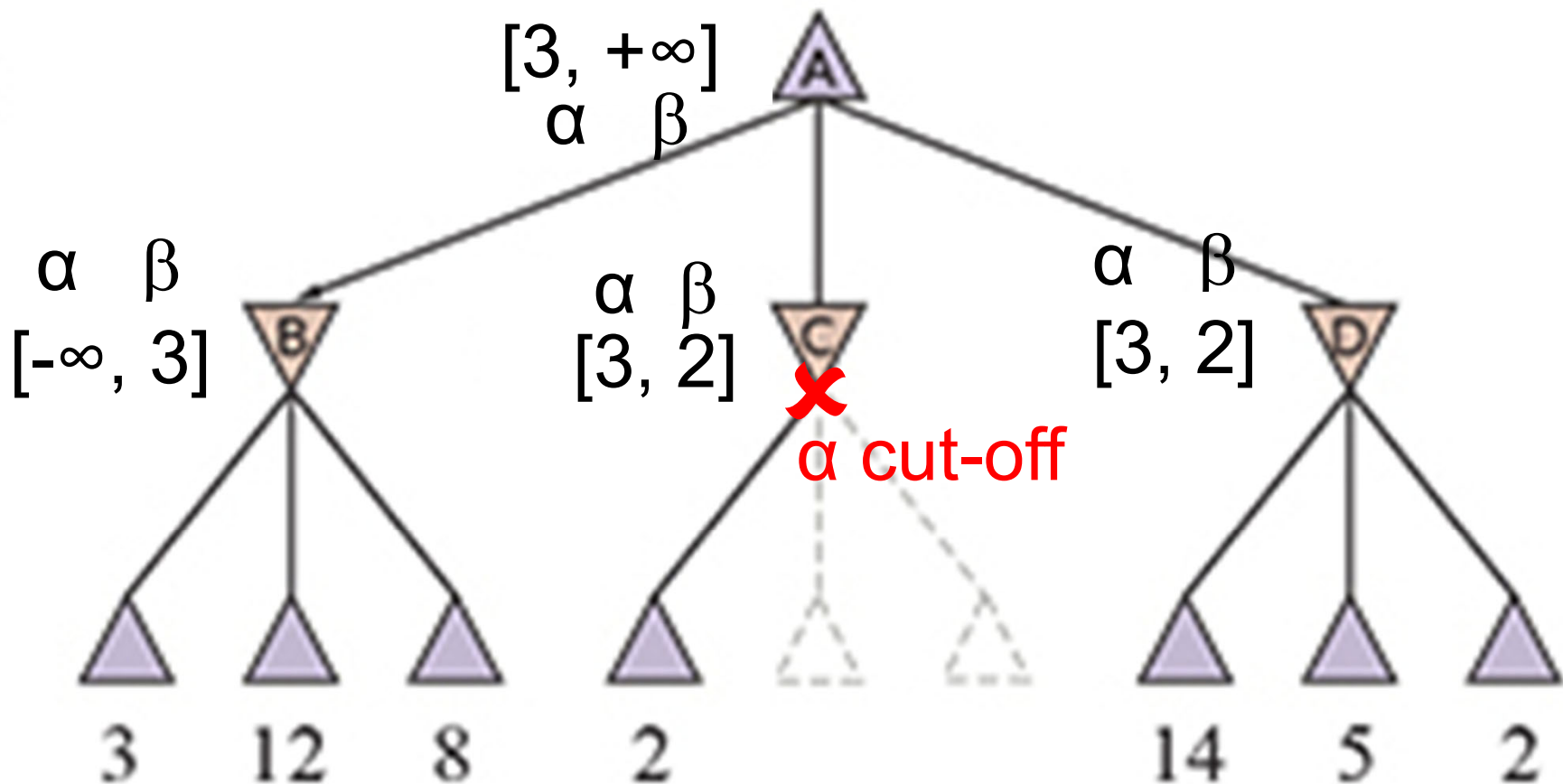


Alpha-Beta Search – Example (V)



Alpha-Beta Search – Example (VI)

(f)



Move Ordering

- **The effectiveness of the $\alpha\beta$ algorithm depends on the order in which states are examined**
- **With perfect ordering, time complexity = $O(b^{m/2})$**
 - depth of search can be doubled
- **Adding dynamic ordering schemes brings us close to the theoretical limit**





FIT3080 – Artificial Intelligence

Resource Limits

Resource Limits

- Suppose we have 100 secs per move, and we explore 10^4 nodes/sec
 - 10^6 nodes per move
 - may not reach a terminal node

Treat the frontier as a terminal node
→ Use an evaluation function



Evaluation Functions

- **From experience**

- E.g., for chess

- > Material value of pieces (pawn = 1, ..., queen = 9)

- > Linear weighted sum of features

- $$\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s),$$

- where

- $$f_1(s) = \{\# \text{ of white queens}\} - \{\# \text{ of black queens}\}$$

- **From pre-processing**

- E.g., for chess, end-game databases (last few plies are pre-computed)



Search Strategies (I)

- **Depth limited search**
 - perhaps add
 - > Quiescence search
 - > Singular extension
- **Iterative Deepening search**
- **Beam search that looks only at n-best moves (forward pruning)**

Search Strategies (II)

- **Type A – wide but shallow**
 - Historically, most chess programs
- **Type B – deep but narrow**
 - Most Go programs



Deterministic Games in Practice

- **Checkers:** Chinook defeated the world champion in an abbreviated game in 1990.
It uses $\alpha\beta$ search combined with a pre-computed database defining perfect play for 39 trillion endgame positions.
- **Chess:** Deep Blue defeated human world champion Garry Kasparov in a six-game match in 1997.
It searches 30 billion positions per move (200 million per second), normally searching to depth 14, and extending the search up to depth 40 for promising options.
Heuristics reduce the EBF to about 3.
- **Othello:** In 1997, a computer defeated the world champion 6-0. Humans are no match for computers.
- **Go:** $b > 361$, which is too large for $\alpha\beta$. In 2016, AlphaGo, which uses Deep Learning, beat the world champion 4-1.

Practice Example

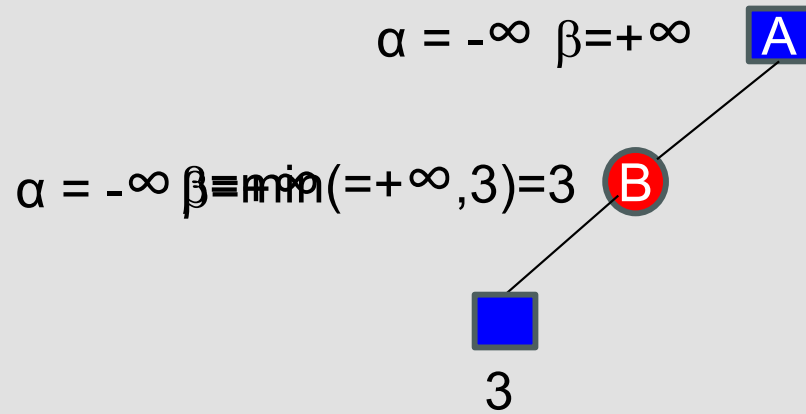
Consider a hypothetical game with branching factor 3. It is MAX's turn to play. They are able to evaluate a position 2 steps in advance. The following is a list of the values of the positions at the bottom of the game tree, should they ever need to be evaluated:

Node #	1	2	3	4	5	6	7	8	9
e(n)	3	5	6	2	9	1	4	7	8

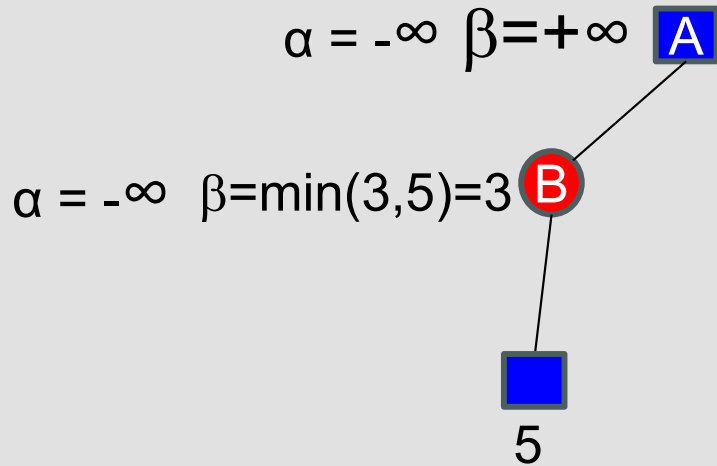
- Draw a step-by-step search tree using the R&N version of α - β pruning.



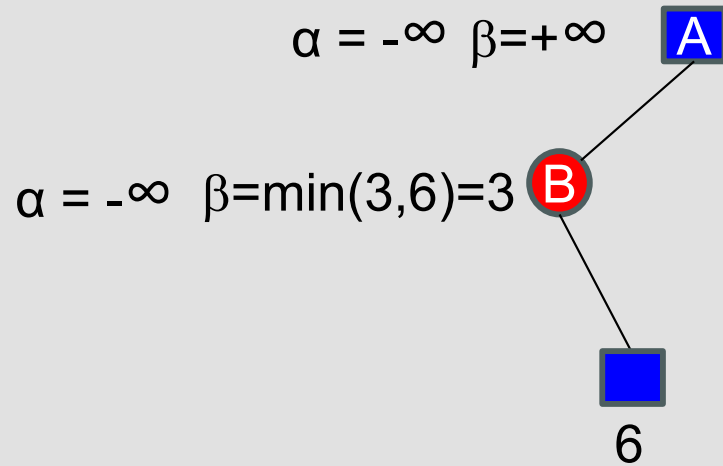
Practice Example (I)



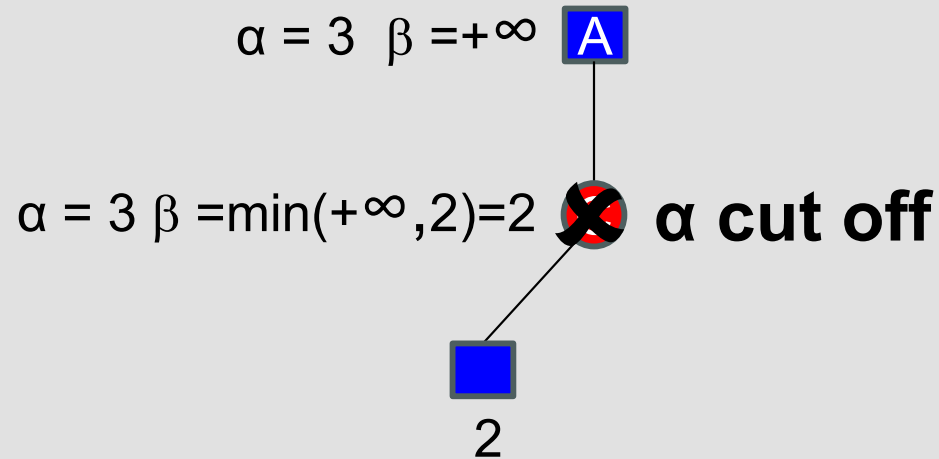
Practice Example (II)



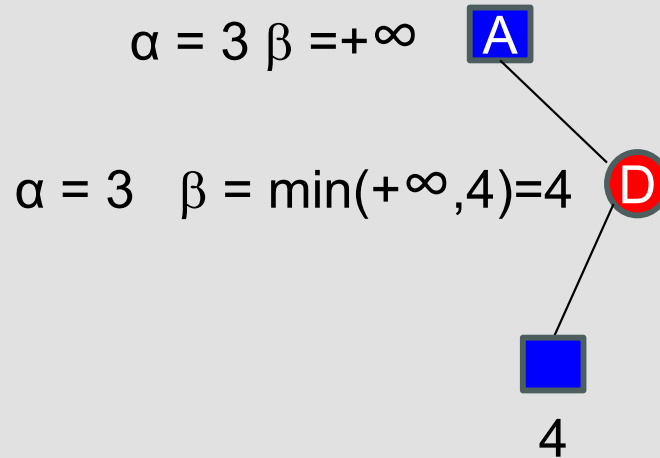
Practice Example (III)



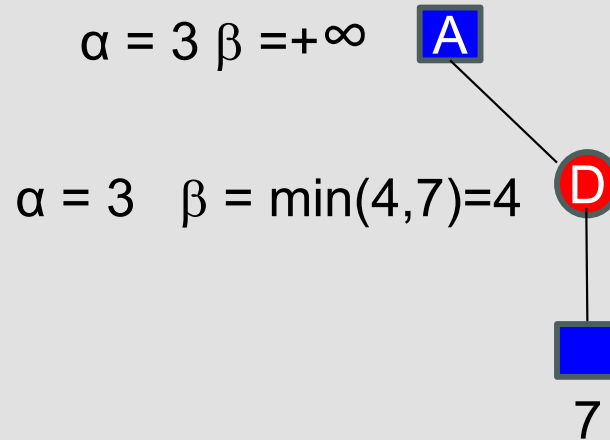
Practice Example (IV)



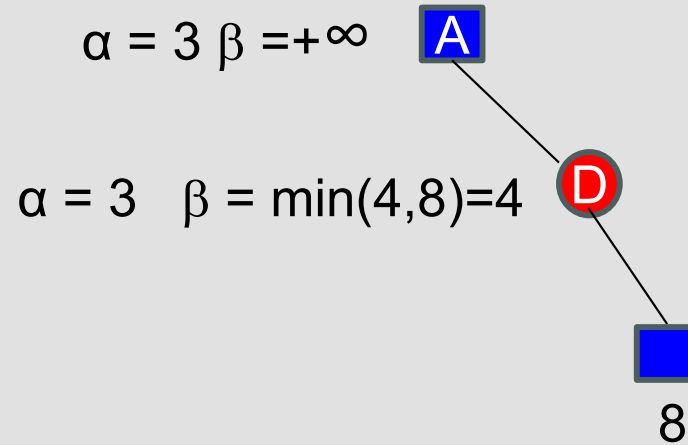
Practice Example (V)



Practice Example (VI)



Practice Example (VII)



Practice Example (VII)

$$\alpha = \max(3,4)=4 \quad \beta =4 \quad \boxed{A}$$

Choose D

Summary: Adversarial Search

Games illustrate important points about AI

- **Perfection is unattainable → must approximate**
- **Force us to think about what to think about, e.g., nodes to keep/discard**



Reading

- **Russell, S. and Norvig, P., *Artificial Intelligence – A Modern Approach* (4th ed), Prentice Hall**
 - Chapter 4, Section 4.1 – irrevocable search algorithms
 - Chapter 6, Sections 6.1-6.3 – adversarial search algorithms



Next Lecture Topic

- **Lecture Topic 4**
 - Propositional and First Order Logic

